

Lecture 9

Behavioral Subtyping

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem
Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Analogy

Scenario: You are the supervisor for the *bidding of government construction projects*, and you want to choose bidders *capable of handling the projects*, and also *able to deliver within budget*

(These scenarios are hypothetical and are not accurate descriptions of actual public bidding processes)

Analogy

The bidders also have preferences in projects they want to take, and want to maximize their profit

(These scenarios are hypothetical and are not accurate descriptions of actual public bidding processes)

Analogy

The contractor St. Bernard Construction has been approved to work on the **Steel Bridge** project, but the project got cancelled . You trust this contractor, so you want to *substitute* a different project for them to do.

Steel Bridge (Project Requirements)

- Steelworking expertise
- 45 meters

Analogy

Stainless Steel Bridge

- Stainless steel expertise
- 45 meters

Iron Bridge

- Ironworking expertise
- 45 meters

Wooden Bridge

- Woodworking expertise
- 45 meters

Which project can you substitute for **Steel Bridge**?

Analogy

Stainless Steel Bridge

- Stainless steel expertise
- 45 meters

Wooden Bridge

- Woodworking expertise
- 45 meters

Iron Bridge

- Ironworking expertise
- 45 meters

Analogy

A certain project Pink Footbridge was originally assigned to St. Bernard Construction, but they cancelled the contract in favor of Iron Bridge. You want the footbridge to be constructed as soon as possible, so you need to *find a replacement contractor*.

Analogy

St. Bernard Construction is an upstanding contractor, and follows bidding processes. When offered a project, they confirm with an *Approval*, which contains:

- status of approval, which is either *Approved*, *Pending*, or *Denied*

You like this kind of no-nonsense approach by St. Bernard, so you expect the substitute contractor to respond like that.

Analogy

The following contractors send approvals in a different way:

Chihuahua Construction

- status of approval, which is either *Approved*, *Pending*, *Denied*, or *Approved with additional budget*

Bulldog Construction

- status of approval, which is either *Approved* or *Pending*

Which contractor can substitute for St. Bernard Construction?

Analogy

Chihuahua Construction

- status of approval, which is either *Approved*, *Pending*, *Denied*, or *Approved with additional budget*

Bulldog Construction

- status of approval, which is either *Approved* or *Pending*

Which contractor can substitute for St. Bernard Construction?

Summary

Guiding principles to keep everyone happy:

Input project to St. Bernard (to substitute)	Can be more lax	...liberal in what they accept
Output to you (from substitute)	Can be stricter	...conservative in what is expected of them

Behavioral Subtyping

Behavioral subtyping: Set of rules a subtype should follow to *meet original expectations of supertype*

Rules (informally):

- “Do not require more than what is originally needed”
- “Do not give less than what is originally promised”
- “Stay true to your roots”

In general: *“Do not subvert anything expected of the original”*

Liskov Substitution Principle

- [Original 1994 paper](#)

Let $\varphi(x)$ be a property provable about objects x of type T .
Then $\varphi(y)$ should be true for objects y of type S where S is a subtype of T .

- “Subtypes should not break behavior expected of their supertype”
- Also known as **behavioral subtyping**
- CS 133: Undecidable problem (Halting Problem reduction)

Liskov Substitution Principle

For inheritance: `Child` must be a behavioral subtype of `Parent`

- If $S <: T$, then `S` objects must behave as how `T` objects are expected to
- A subclass must uphold all contracts stipulated by its superclass

`Child` is able to *flawlessly masquerade* as a `Parent` by meeting all client expectations

Behavioral Subtyping as Impersonation



Behavioral Subtyping as Impersonation



LSP Rules

Type Rules

1. *Method parameters* must be contravariant
2. *Method return types* must be covariant
3. *Exceptions thrown* must be subtypes of original

Behavioral Rules

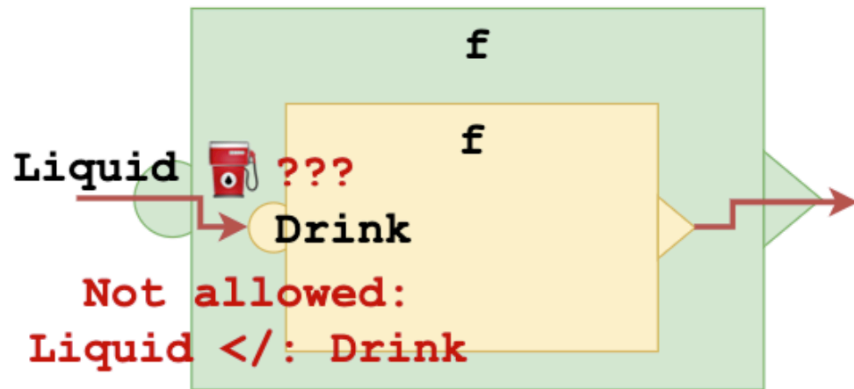
4. *Preconditions* must not be strengthened
5. *Postconditions* must not be weakened
6. *Invariants* must not be violated
7. The *history rule* must not be violated

1. Method Parameters Must Be Contravariant

Given: `Drink <: Liquid`

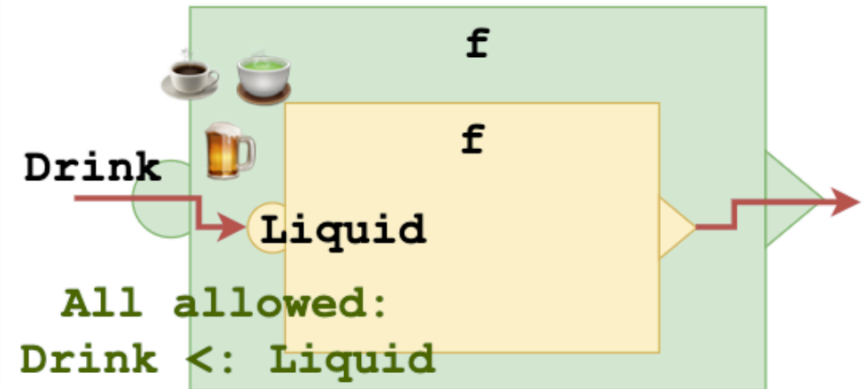
Covariance:

`f(x: Drink) <: f(x: Liquid)`



Contravariance:

`f(x: Liquid) <: f(x: Drink)`



1. Method Parameters Must Be Contravariant

- Informal rule: “Do *not* require *stricter* than what is originally needed”

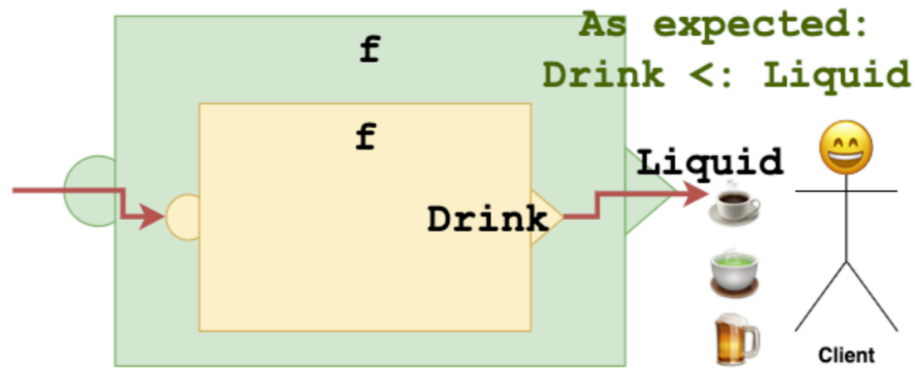
09/method_parameters_contravariant

2. Method Return Types Must Be Covariant

Given: `Drink <: Liquid`

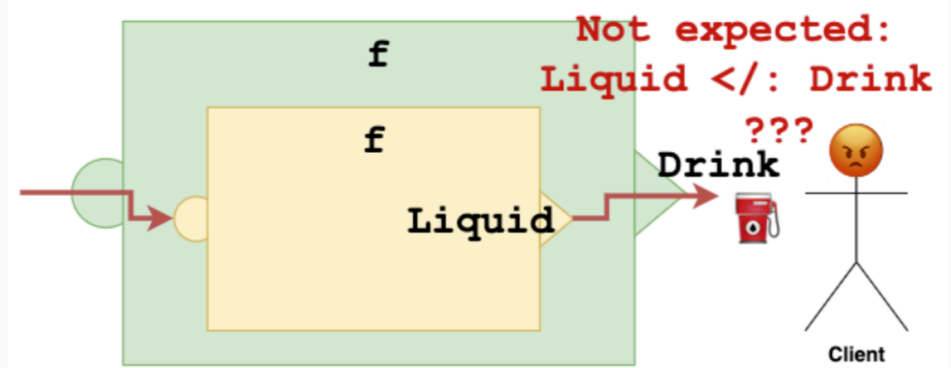
Covariance:

`f() -> Drink <: f() -> Liquid`



Contravariance:

`f() -> Liquid <: f() -> Drink`



2. Method Return Types Must Be Covariant

- Informal rule: “Do *not* give *outside of* what was originally promised”

09/method_return_types_covariant

3. Exceptions Thrown Must Be Subtypes of Original

- Python Exception Subclass Hierarchy

<https://docs.python.org/3/library/exceptions.html>

```
└─ Exception
    └─ ArithmeticError
        └─ FloatingPointError
        └─ OverflowError
        └─ ZeroDivisionError
    └─ AssertionError
    └─ AttributeError
    └─ BufferError
    └─ EOFError
    └─ ExceptionGroup [BaseExceptionGroup]
    └─ ImportError
        └─ ModuleNotFoundError
    └─ LookupError
        └─ IndexError
        └─ KeyError
    └─ MemoryError
    └─ NameError
        └─ UnboundLocalError
    └─ OSError
```


3. Exceptions Thrown Must Be Subtypes of Original

09/exceptions

4. Preconditions Must Not Be Strengthened

Precondition: Condition that must be satisfied *before* a function is called

- “Strengthen”: Become more narrow or less permissive
- Normally applies to arguments and/or object state
- Preconditions must be stated in function’s documentation

4. Preconditions Must Not Be Strengthened

Precondition: Condition that must be satisfied *before* a function is called

- Informal rule: “Do *not* require *stricter than* what is *originally needed*”

09/preconditions

4. Preconditions Must Not Be Strengthened

- `.sqrt(...)` of `MoreEfficientMathUtils` is more demanding of its argument than overridden `.sqrt(...)`; **LSP violation!**
- Precondition of `.sqrt(...)` for `MoreEfficientMathUtils` is violated with `1.5`; will likely give an *incorrect answer* or *crash outright*

5. Postconditions Must Not Be Weakened

Postcondition: Condition that must be satisfied *after* a function is called

- “Weaken”: Become wider or more permissive
- Normally applies to return value and/or object state
- Postconditions must be stated in function’s documentation
- Includes raising of exceptions

5. Postconditions Must Not Be Weakened

Postcondition: Condition that must be satisfied *after* a function is called

- Informal rule: “Do *not* give *outside of* what was *originally promised*”

09/postconditions

5. Postconditions Must Not Be Weakened

- `.random(...)` of `DeckManagerV2B` is *more lax with its return value* than overridden `.random(...)`; **LSP violation!**
- Nonempty postcondition of `.random(...)` of `DeckManagerV1` is not honored by `DeckManagerV2B`; may give an *incorrect answer* or *crash outright* for dependent clients

6. Invariants Must Not Be Violated

Invariant: Condition that is true throughout the entire lifetime of object

- *Child* must never violate invariants maintained by *Parent*

09/invariants

6. Invariants Must Not Be Violated

- Client depends on the *immutability invariant* stated in the docstring of `ImmutablePoint` which is violated by `HalfMutablePoint`; Client computations *might be erroneous* when using it!

6. Invariants Must Not Be Violated

- Informal rule: “Stay true to your *roots*”
- For simplicity, invariants must be stated explicitly for them to be considered

09/invariants2

6. Invariants Must Not Be Violated

- No invariant violation (but in practice, you could argue that the names sufficiently specify the bounded invariant)

6. Invariants Must Not Be Violated

- Informal rule: “Stay true to your *roots*”
- For simplicity, invariants must be stated explicitly for them to be considered

09/invariants3

- Stated invariant is *violated*

7. History Rule Must Not Be Violated

History rule: Any possible state transitions of *Child* as seen through *Parent* methods should be attainable using only methods of *Parent*

- Parent-based state changes in Child must be possible for Parent as well
- Informal rule: “Stay true to your roots”

09/history_rule

7. History Rule Must Not Be Violated

- Violates invariant of `IntStack`

Behavioral Subtyping Summary

	Type Rule	Behavioral Rule
"Do not require stricter than what is originally needed"	• Method parameters must be contravariant	• Preconditions must not be strengthened
"Do not give outside of what was originally promised"	• Method return types must be covariant • Exceptions thrown must be	• Postconditions must not be weakened

	Type Rule	Behavioral Rule
	subtypes of original	

	Type Rule	Behavioral Rule
"Stay true to your roots"	-	• Invariants must not be violated • History rule must not be violated